

Bài 3: Câu lệnh rẽ nhánh

Khoa Khoa học Dữ liệu và Trí tuệ Nhân tạo (FDA) - Đại học Kinh tế Quốc dân

Mã học phần: DSAI1003

Contents

Bài 3. Câu lệnh rẽ nhánh	4
Giới thiệu bài học	4
Kiến thức và kỹ năng cần đạt	5
Cấu trúc bài học	5
3.1 Câu lệnh if	6
3.1.1 Quyết định hai nhánh	6
3.1.2 Cú pháp tổng quát	6
3.1.3 Câu lệnh ghép và khối lệnh	7
Ví dụ – Mô phỏng thang máy	7
Lỗi thường gặp 3.1 – Tab và thụt lề không nhất quán	7
Mẹo lập trình 3.1 – Tránh lặp mã trong các nhánh	8
Chủ đề bổ sung 3.1 – Biểu thức điều kiện	8
Tự kiểm tra 3.1	8
Câu 1	8
Câu 2	9
Câu 3	9
Câu 4	9
3.2 Các toán tử quan hệ	9
3.2.1 Các toán tử quan hệ	9
3.2.2 Phép gán = và phép kiểm tra bằng ==	10
3.2.3 So sánh chuỗi	10
3.2.4 Thứ tự ưu tiên toán tử	10
Lỗi thường gặp 3.2 – So sánh chính xác số thực dấu phẩy động	11
Chủ đề bổ sung 3.2 – Thứ tự từ điển của chuỗi	11
CÁCH LÀM 3.1 – Cài đặt một câu lệnh if	12
Bước 1. Xác định điều kiện rẽ nhánh	12
Bước 2. Mô tả điều xảy ra khi điều kiện đúng	12
Bước 3. Mô tả điều xảy ra khi điều kiện sai	12
Bước 4. Kiểm tra toán tử quan hệ tại giá trị biên	12
Bước 5. Loại bỏ phần xử lý lặp lại	12
Bước 6. Kiểm thử cả hai nhánh	12
Bước 7. Ghép thành chương trình Python	13
Ví dụ minh họa 3.1 – Lấy phần giữa của một chuỗi	13

Tự kiểm tra 3.2	14
Câu 1	14
Câu 2	14
Câu 3	14
Câu 4	14
3.3 Rẽ nhánh lồng nhau	14
3.3.1 Quyết định nhiều tầng	15
Mẹo lập trình 3.2 – Mô phỏng bằng tay	15
Máy tính và xã hội 3.1 – Hệ thống phức tạp và logic quyết định	16
Tự kiểm tra 3.3	16
Câu 1	16
Câu 2	16
Câu 3	16
3.4 Nhiều phương án	16
Ví dụ – Phân loại động đất	17
3.4.1 Thứ tự điều kiện rất quan trọng	17
3.4.2 elif và các câu lệnh if độc lập	18
Tự kiểm tra 3.4	18
Câu 1	18
Câu 2	18
TOOLBOX 3.1 – Gửi e-mail	19
3.5 Giải quyết vấn đề: Lưu đồ	19
3.5.1 Quyết định hai nhánh	19
3.5.2 Nhiều phương án	19
3.5.3 Tránh luồng điều khiển kiểu “spaghetti”	20
Ví dụ – Chi phí vận chuyển	20
Tự kiểm tra 3.5	20
Câu 1	20
Câu 2	20
3.6 Giải quyết vấn đề: Ca kiểm thử	21
3.6.1 Bao phủ nhánh	21
3.6.2 Các giá trị biên	21
3.6.3 Thiết kế ca kiểm thử trước khi lập trình	21
Mẹo lập trình 3.3 – Dành thời gian cho kiểm thử và gỡ lỗi	22
Tự kiểm tra 3.6	22
Câu 1	22
Câu 2	22
3.7 Biến Boolean và các toán tử Boolean	22
3.7.1 Toán tử and	23
3.7.2 Toán tử or	23
3.7.3 Toán tử not	23
3.7.4 Thứ tự ưu tiên	24
Lỗi thường gặp 3.3 – Nhầm and với or	24
Mẹo lập trình 3.4 – Tính dễ đọc	24

Chủ đề bổ sung 3.3 – Nối chuỗi các phép so sánh	25
Chủ đề bổ sung 3.4 – Đánh giá ngắn mạch	25
Chủ đề bổ sung 3.5 – Định luật De Morgan	25
Tự kiểm tra 3.7	26
Câu 1	26
Câu 2	26
Câu 3	26
Câu 4	26
3.8 Phân tích chuỗi	27
3.8.1 Toán tử thành viên in và not in	27
3.8.2 Một số phương thức chuỗi con hữu ích	27
3.8.3 Kiểm tra đặc điểm của chuỗi	27
Ví dụ – Khảo sát chuỗi con	28
Tự kiểm tra 3.8	28
Câu 1	28
Câu 2	28
Câu 3	29
3.9 Ứng dụng: Kiểm tra dữ liệu đầu vào	29
3.9.1 Kiểm tra miền giá trị	29
3.9.2 Kiểm tra lựa chọn nhập vào	30
3.9.3 Giới hạn ở giai đoạn hiện tại	30
Chủ đề bổ sung 3.6 – Kết thúc chương trình	31
Chủ đề bổ sung 3.7 – Chương trình đồ họa tương tác	31
Máy tính và xã hội 3.2 – Trí tuệ nhân tạo	31
Ví dụ minh họa 3.2 – Hai đường tròn giao nhau	31
TOOLBOX 3.2 – Vẽ đồ thị đơn giản	32
Tổng kết Bài 3	33
Câu lệnh if	33
Các toán tử quan hệ	33
Rẽ nhánh lồng nhau	33
Nhiều phương án	33
Lưu đồ	33
Ca kiểm thử	34
Logic Boolean	34
Phân tích chuỗi	34
Kiểm tra dữ liệu đầu vào	34
Quiz tổng kết	35
Câu 1	35
Câu 2	35
Câu 3	35
Câu 4	35
Câu 5	36
Câu 6	36
Câu 7	36
Câu 8	36

Câu 9	36
Câu 10	37
Câu 11	37
Câu 12	37

Bài tập ôn tập **37**

Bài 1. Theo dõi quyết định hai nhánh	37
Bài 2. Chọn đúng toán tử quan hệ	37
Bài 3. Viết lại để tránh lặp mã	38
Bài 4. Thiết kế ca kiểm thử	38
Bài 5. Biểu thức Boolean	38
Bài 6. Phân tích chuỗi	39

Bài tập thực hành **39**

Bài 1. Đạt / Không đạt / Điểm không hợp lệ	39
Bài 2. Số lớn hơn trong hai số	39
Bài 3. Số lớn nhất trong ba số	39
Bài 4. Quy tắc vận chuyển	39
Bài 5. Phân loại điểm chữ	40
Bài 6. Kiểm tra tên người dùng	40
Bài 7. Phân loại loại file	40
Bài 8. So sánh gần bằng	40
Bài 9. Phân loại tam giác theo độ dài cạnh	40
Bài 10. Kiểm tra mã truy cập đơn giản	41

Bài tập mở **41**

Bài 1	41
Bài 2	41
Bài 3	41
Bài 4	41
Bài 5	42

Thuật ngữ chính **42**

Hướng dẫn học tập **42**

Bài 3. Câu lệnh rẽ nhánh

Cập nhật lần cuối: 19/09/2026

Nguồn: Chương 3 – *Decisions*, trong giáo trình *Python for Everyone* của Cay Horstmann và Rance Necaise (2/e).

Bài giảng này giữ cấu trúc, thuật ngữ, ví dụ và trọng tâm sự phạm của Chương 3, đồng thời sắp xếp lại nội dung để thuận tiện cho giảng dạy trực tiếp trên lớp. Các ví dụ vẫn theo phong cách đặt tên biến và định dạng chuỗi bằng toán tử % như trong giáo trình.

Giới thiệu bài học

Ở Bài 2, phần lớn các chương trình được thực hiện theo thứ tự từ trên xuống dưới. Cách thực thi này phù hợp với các phép tính trực tiếp, nhưng các chương trình thực tế còn cần khả năng **đưa ra quyết định**.

Một quyết định cho phép chương trình chọn hành động khác nhau tùy theo dữ liệu hoặc hoàn cảnh. Ví dụ, chương trình có thể cần:

- áp dụng mức giảm giá khác nhau cho đơn hàng nhỏ và đơn hàng lớn;
- từ chối số tầng không hợp lệ trong chương trình mô phỏng thang máy;
- phân loại động đất theo độ lớn;
- kiểm tra một chuỗi có đúng định dạng yêu cầu hay không;
- chọn công thức tính thuế theo tình trạng hôn nhân và thu nhập;
- kiểm tra đồng thời nhiều điều kiện.

Ý tưởng trung tâm của bài học là:

Đánh giá một điều kiện

↓

Đúng hay Sai?

↙

↘

Hành động A Hành động B

↘

↙

Tiếp tục chương trình

Nội dung được phát triển từ câu lệnh `if` hai nhánh, sau đó mở rộng sang rẽ nhánh lồng nhau, nhiều phương án, biểu thức Boolean, phân tích chuỗi, thiết kế ca kiểm thử và kiểm tra dữ liệu đầu vào.

Kiến thức và kỹ năng cần đạt

Sau khi hoàn thành bài học này, sinh viên có thể:

- Sử dụng các cấu trúc `if`, `if/else` và `if/elif/else`.
- Giải thích vai trò của **điều kiện** trong luồng điều khiển của chương trình.
- Sử dụng thuật ngữ đúng để xác định các khối lệnh.
- So sánh số và chuỗi bằng các toán tử quan hệ.
- Phân biệt phép gán `=` với phép kiểm tra bằng nhau `==`.
- Tránh so sánh bằng tuyệt đối với các số thực được tạo ra từ phép tính khi cần kiểm tra gần bằng.
- Thiết kế quyết định hai nhánh từ mô tả bài toán.
- Cài đặt **rẽ nhánh lồng nhau** cho các quyết định nhiều tầng.
- Sử dụng `elif` cho các phương án loại trừ lẫn nhau.
- Sắp xếp điều kiện theo thứ tự phù hợp, đặc biệt với các ngưỡng chồng lấn.
- Đọc và xây dựng **lưu đồ** đơn giản.
- Thiết kế **ca kiểm thử** bao phủ các nhánh và giá trị biên.
- Sử dụng hai giá trị Boolean `True` và `False`.
- Kết hợp điều kiện bằng `and`, `or` và `not`.
- Giải thích thứ tự ưu tiên và cơ chế đánh giá ngắn mạch.
- Áp dụng định luật De Morgan để biến đổi biểu thức logic.
- Phân tích chuỗi bằng toán tử thành viên và các phương thức chuỗi.
- Kiểm tra tính hợp lệ của dữ liệu đầu vào trước khi xử lý.
- Chuẩn hóa dữ liệu văn bản bằng `upper()` hoặc `lower()` khi không cần phân biệt chữ hoa/chữ thường.

Cấu trúc bài học

1. Câu lệnh `if`
2. Các toán tử quan hệ
3. Rẽ nhánh lồng nhau
4. Nhiều phương án
5. Giải quyết vấn đề: Lưu đồ
6. Giải quyết vấn đề: Ca kiểm thử
7. Biến Boolean và toán tử Boolean

8. Phân tích chuỗi
 9. Ứng dụng: Kiểm tra dữ liệu đầu vào
 10. Các chủ đề và Toolbox tùy chọn
 11. Tổng kết và bài tập
-

3.1 Câu lệnh if

Chương trình thường cần thực hiện một hành động khi điều kiện đúng và một hành động khác khi điều kiện sai. Câu lệnh if cho phép chương trình thực hiện việc rẽ nhánh này.

3.1.1 Quyết định hai nhánh

Xét hệ thống thang máy trong một tòa nhà bỏ qua số tầng 13. Khi người dùng chọn tầng hiển thị lớn hơn 13, tầng vật lý thực tế thấp hơn 1 đơn vị. Ví dụ, tầng hiển thị 20 tương ứng với tầng vật lý 19.

Ta có thể viết:

```
if floor > 13 :
    actualFloor = floor - 1
else :
    actualFloor = floor
```

Về mặt ý tưởng:

```

                floor > 13 ?
                /         \
            Đúng          Sai
            /             \
actualFloor = floor - 1   actualFloor = floor
```

Chỉ **một nhánh** được thực hiện.

Nếu `floor > 13` là đúng, Python thực hiện khối lệnh đầu tiên và bỏ qua khối `else`. Nếu điều kiện sai, Python bỏ qua khối đầu và thực hiện nhánh `else`.

3.1.2 Cú pháp tổng quát

```
if condition :
    statements1
else :
    statements2
```

Nhánh `else` là tùy chọn:

```
if condition :
    statements
```

Ví dụ:

```
actualFloor = floor
```

```
if floor > 13 :
    actualFloor = actualFloor - 1
```

Cách viết này phù hợp khi trường hợp sai không cần hành động đặc biệt nào.

3.1.3 Câu lệnh ghép và khối lệnh

Câu lệnh `if` là một **câu lệnh ghép**. Dòng đầu của nó kết thúc bằng dấu hai chấm `:`, và các câu lệnh thuộc cùng nhánh phải được thụt lề.

```
if totalSales > 100.0 :
    discount = totalSales * 0.05
    totalSales = totalSales - discount
    print("You received a discount of", discount)
```

Ba dòng được thụt lề tạo thành một khối lệnh.

Các quy tắc quan trọng:

- Dòng tiêu đề kết thúc bằng `:`.
- Các câu lệnh trong cùng một khối phải thụt lề ở cùng mức.
- `if` và `else` tương ứng phải thẳng hàng.
- Khối lệnh kết thúc khi mức thụt lề quay lại mức bên ngoài.

Trong Python, thụt lề là một phần của cú pháp, không chỉ để trình bày đẹp hơn.

Ví dụ – Mô phỏng thang máy

```
##
# This program simulates an elevator panel that skips floor 13.
#

floor = int(input("Floor: "))

if floor > 13 :
    actualFloor = floor - 1
else :
    actualFloor = floor

print("The elevator will travel to the actual floor", actualFloor)
```

Ví dụ chạy chương trình:

```
Floor: 20
The elevator will travel to the actual floor 19
```

Lỗi thường gặp 3.1 – Tab và thụt lề không nhất quán

Python yêu cầu các câu lệnh trong cùng một khối phải có mức thụt lề nhất quán.

Một đoạn mã nhìn có vẻ thẳng hàng trong trình soạn thảo vẫn có thể gặp lỗi nếu trộn ký tự tab và khoảng trắng.

Nên cấu hình trình soạn thảo để phím Tab chèn khoảng trắng. Một quy ước phổ biến là dùng bốn khoảng trắng cho mỗi mức thụt lề:

```
if score >= 50 :
    print("Pass")
    print("Continue to the next course")
```

Điều quan trọng nhất là **tính nhất quán**.

Mẹo lập trình 3.1 – Tránh lặp mã trong các nhánh

Xét đoạn mã:

```
if floor > 13 :
    actualFloor = floor - 1
    print("Actual floor:", actualFloor)
else :
    actualFloor = floor
    print("Actual floor:", actualFloor)
```

Câu lệnh print bị lặp lại ở cả hai nhánh.

Cách viết tốt hơn:

```
if floor > 13 :
    actualFloor = floor - 1
else :
    actualFloor = floor

print("Actual floor:", actualFloor)
```

Đưa phần xử lý chung ra khỏi các nhánh giúp:

- giảm lặp mã;
 - chương trình ngắn hơn;
 - giảm nguy cơ sửa một bản nhưng quên sửa bản còn lại.
-

Chủ đề bổ sung 3.1 – Biểu thức điều kiện

Python có thể biểu diễn một lựa chọn hai trường hợp dưới dạng biểu thức:

```
actualFloor = floor - 1 if floor > 13 else floor
```

Tương đương với:

```
if floor > 13 :
    actualFloor = floor - 1
else :
    actualFloor = floor
```

Ở giai đoạn đầu học lập trình, dạng if/else đầy đủ thường dễ đọc hơn, đặc biệt khi mỗi nhánh có nhiều hơn một hành động.

Tự kiểm tra 3.1

Câu 1

Giá trị của actualFloor là bao nhiêu khi floor = 18?

```
if floor > 13 :
    actualFloor = floor - 1
```

```
else :  
    actualFloor = floor
```

Đáp án

17.

Câu 2

Giá trị của actualFloor là bao nhiêu khi floor = 9?

Đáp án

9.

Câu 3

Tại sao phải có dấu : sau điều kiện của if?

Đáp án

Dấu : đánh dấu dòng tiêu đề của câu lệnh ghép. Khối lệnh được thực l  ngay sau đó thuộc về dòng tiêu đề này.

Câu 4

Có thể cải thiện đoạn mã sau như thế nào?

```
if balance < 0 :  
    status = "Overdrawn"  
    print(status)  
else :  
    status = "OK"  
    print(status)
```

Đáp án

Đưa câu lệnh print ra ngoài hai nhánh:

```
if balance < 0 :  
    status = "Overdrawn"  
else :  
    status = "OK"  
  
print(status)
```

3.2 Các toán tử quan hệ

Một câu lệnh if cần một điều kiện có giá trị là True hoặc False.

Nhiều điều kiện được tạo bằng cách so sánh hai giá trị với **toán tử quan hệ**.

3.2.1 Các toán tử quan hệ

Toán tử Python	Ý nghĩa
>	Lớn hơn
>=	Lớn hơn hoặc bằng
<	Nhỏ hơn

Toán tử Python	Ý nghĩa
<=	Nhỏ hơn hoặc bằng
==	Bằng
!=	Khác

Ví dụ:

```
3 < 4
4 <= 4
7 != 5
10 == 2 * 5
```

Mỗi biểu thức trên cho kết quả là một giá trị Boolean.

3.2.2 Phép gán = và phép kiểm tra bằng ==

Hai toán tử này có chức năng hoàn toàn khác nhau.

Phép gán:

```
floor = 13
```

Phép kiểm tra bằng nhau:

```
if floor == 13 :
    print("Floor 13 selected")
```

Có thể đọc như sau:

= → gán
 == → có bằng nhau không?

3.2.3 So sánh chuỗi

Chuỗi cũng có thể được so sánh.

```
name1 = "John Wayne"
name2 = "John Wayne"
```

```
if name1 == name2 :
    print("The strings are identical.")
```

So sánh chuỗi phân biệt chữ hoa và chữ thường:

```
"John" == "john"
```

cho kết quả False.

Để hai chuỗi bằng nhau, mọi ký tự và vị trí của ký tự phải trùng khớp.

3.2.4 Thứ tự ưu tiên toán tử

Các toán tử số học có độ ưu tiên cao hơn toán tử quan hệ.

```
floor - 1 < 13
```

Python tính trước:

```
floor - 1
```

sau đó mới so sánh kết quả với 13.

Với các biểu thức phức tạp, nên dùng dấu ngoặc để tăng tính dễ đọc.

Lỗi thường gặp 3.2 – So sánh chính xác số thực dấu phẩy động

Các phép tính với float có thể xuất hiện sai số làm tròn rất nhỏ.

Ví dụ:

```
from math import sqrt
```

```
x = sqrt(2.0)
print(x * x)
```

Kết quả có thể rất gần 2.0, nhưng biểu diễn bên trong không nhất thiết chính xác tuyệt đối bằng 2.0.

Vì vậy phép kiểm tra sau có thể không phù hợp:

```
if x * x == 2.0 :
    print("Exactly two")
```

Khi cần kiểm tra gần bằng, so sánh độ chênh với một ngưỡng nhỏ:

```
EPSILON = 1E-14
```

```
if abs(x * x - 2.0) < EPSILON :
    print("Approximately two")
```

Mẫu tổng quát:

```
if abs(x - y) < epsilon :
    # Coi x và y là gần bằng nhau.
```

Giá trị epsilon phải được chọn phù hợp với thang đo và mục đích của phép tính.

Chủ đề bổ sung 3.2 – Thứ tự từ điển của chuỗi

Các toán tử như < và > cũng có thể dùng với chuỗi.

Python so sánh chuỗi theo thứ tự từ điển dựa trên mã ký tự.

Ví dụ, chữ hoa/chữ thường có ảnh hưởng:

```
"John" < "john"
```

Kết quả dựa trên thứ tự mã ký tự, không phải quy tắc từ điển bỏ qua chữ hoa/chữ thường.

Nếu muốn so sánh không phân biệt hoa thường, có thể chuẩn hóa trước:

```
name1 = name1.lower()
name2 = name2.lower()
```

sau đó mới so sánh.

CÁCH LÀM 3.1 – Cài đặt một câu lệnh if

Chương đưa ra một quy trình có hệ thống để xây dựng quyết định.

Giả sử một cửa hàng áp dụng một mức giảm giá cho giá bán dưới một ngưỡng và mức giảm lớn hơn khi đạt hoặc vượt ngưỡng đó.

Bước 1. Xác định điều kiện rẽ nhánh

Đặt một câu hỏi có thể trả lời bằng True hoặc False.

Ví dụ:

Giá ban đầu có nhỏ hơn 128 không?

Bước 2. Mô tả điều xảy ra khi điều kiện đúng

Dùng mức giảm giá thấp hơn.

Bước 3. Mô tả điều xảy ra khi điều kiện sai

Dùng mức giảm giá cao hơn.

Bước 4. Kiểm tra toán tử quan hệ tại giá trị biên

Nếu mức giảm cao hơn bắt đầu ngay tại 128, điều kiện cho mức thấp phải là:

```
originalPrice < 128
```

không phải:

```
originalPrice <= 128
```

Các giá trị biên thường giúp phát hiện lỗi chọn < hay <=.

Bước 5. Loại bỏ phần xử lý lặp lại

Thay vì tính toàn bộ giá sau giảm trong cả hai nhánh, chỉ chọn hệ số giảm giá bên trong nhánh:

```
if originalPrice < 128 :  
    discountRate = 0.92  
else :  
    discountRate = 0.84
```

```
discountedPrice = discountRate * originalPrice
```

Bước 6. Kiểm thử cả hai nhánh

Chọn ít nhất một giá trị dưới ngưỡng và một giá trị trên ngưỡng.

Ví dụ:

100 → nhánh giảm thấp hơn
 200 → nhánh giảm cao hơn
 128 → giá trị biên

Bước 7. Ghép thành chương trình Python

```
originalPrice = float(input("Original price before discount: "))
```

```
if originalPrice < 128 :
    discountRate = 0.92
else :
    discountRate = 0.84
```

```
discountedPrice = discountRate * originalPrice
print("Discounted price: %.2f" % discountedPrice)
```

Điều quan trọng không phải ghi nhớ ví dụ cụ thể này, mà là biết cách chuyển một quy tắc hai trường hợp thành một điều kiện đúng và hai nhánh tương ứng.

Ví dụ minh họa 3.1 – Lấy phần giữa của một chuỗi

Giả sử chương trình cần lấy:

- một ký tự ở giữa nếu độ dài chuỗi là lẻ;
- hai ký tự ở giữa nếu độ dài chuỗi là chẵn.

Ví dụ:

```
"crate" → "a"
"crates" → "at"
```

Điều kiện rẽ nhánh dựa vào tính chẵn/lẻ:

```
len(text) % 2 == 1
```

Vị trí giữa có thể tính một lần:

```
position = len(text) // 2
```

Sau đó:

```
if len(text) % 2 == 1 :
    result = text[position]
else :
    result = text[position - 1] + text[position]
```

Ví dụ này kết hợp:

- độ dài chuỗi;
 - chia lấy phần nguyên;
 - phép chia lấy dư;
 - truy cập bằng chỉ số;
 - quyết định hai nhánh;
 - tránh tính toán lặp lại.
-

Tự kiểm tra 3.2

Câu 1

Giá trị của:

```
4 <= 4
```

là gì?

Đáp án

True.

Câu 2

Tại sao đoạn sau sai?

```
if score = 100 :  
    print("Perfect")
```

Đáp án

= là phép gán. Muốn kiểm tra bằng nhau phải dùng ==.

Câu 3

Điều kiện nào kiểm tra n là số lẻ?

Đáp án

```
n % 2 == 1
```

Câu 4

Kết quả phần giữa của:

```
text = "monitor"
```

là gì?

Đáp án

"i".

3.3 Rẽ nhánh lồng nhau

Đôi khi chương trình phải đưa ra một quyết định, rồi tiếp tục đưa ra một quyết định khác bên trong nhánh đã được chọn.

Đó là **rẽ nhánh lồng nhau**.

Cấu trúc ví dụ:

```
if condition1 :  
    if condition2 :  
        statementA  
    else :  
        statementB  
else :  
    statementC
```

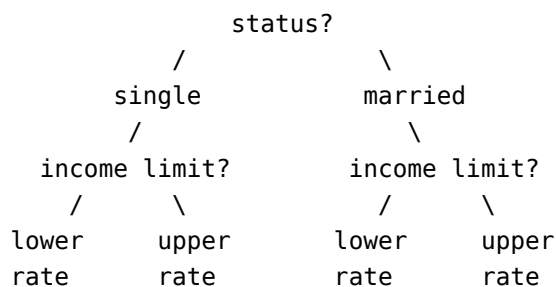
Câu lệnh `if` thứ hai nằm bên trong nhánh đầu của `if` bên ngoài.

3.3.1 Quyết định nhiều tầng

Một ví dụ điển hình là phép tính thuế đơn giản hóa:

1. trước tiên chương trình xác định nhóm người nộp thuế;
2. sau đó xác định mức thu nhập áp dụng trong nhóm đó.

Về mặt ý tưởng:



Mẫu cài đặt:

```

if status == "s" :
    if income <= SINGLE_LIMIT :
        # mức thuế thấp cho người độc thân
        ...
    else :
        # mức thuế cao cho người độc thân
        ...
else :
    if income <= MARRIED_LIMIT :
        # mức thuế thấp cho người đã kết hôn
        ...
    else :
        # mức thuế cao cho người đã kết hôn
        ...
  
```

Rẽ nhánh có thể lồng sâu hơn, nhưng quá nhiều mức lồng nhau làm chương trình khó đọc. Ở các bài sau, hàm sẽ giúp tổ chức logic phức tạp tốt hơn.

Mẹo lập trình 3.2 – Mô phỏng bằng tay

Mô phỏng bằng tay (*hand-tracing*) là thực hiện chương trình thủ công, từng câu lệnh một.

Có thể tạo bảng nhỏ như sau:

Bước	income	status	tax1	tax2
Nhập	80000	"m"		
Khởi tạo	80000	"m"	0	0
Rẽ nhánh	80000	"m"	...	...

Mỗi khi một biến thay đổi, cập nhật giá trị của biến trong bảng.

Kỹ thuật này đặc biệt hữu ích khi:

- có nhiều if lồng nhau;
 - chương trình có nhiều biến;
 - cần hiểu vì sao chương trình đi vào một nhánh nào đó;
 - cần so sánh kết quả mong đợi với kết quả thực tế.
-

Máy tính và xã hội 3.1 – Hệ thống phức tạp và logic quyết định

Chương sử dụng hệ thống xử lý hành lý của Sân bay Quốc tế Denver như một ví dụ về độ phức tạp của phần mềm và hệ thống. Một logic quyết định có thể rất đơn giản khi xét riêng lẻ, nhưng trở nên khó quản lý khi kết hợp nhiều thành phần vật lý, ràng buộc thời gian, trường hợp ngoại lệ và tương tác giữa các thành phần.

Bài học thực tế cho người mới học lập trình là:

Mỗi quyết định cần đủ rõ để có thể hiểu và kiểm thử. Không nên giả định rằng một hệ thống lớn sẽ đúng chỉ vì từng mảnh nhỏ của nó đều có vẻ đúng khi xét riêng.

Tự kiểm tra 3.3

Câu 1

Khi nào nên dùng rẽ nhánh lồng nhau?

Đáp án

Khi quyết định thứ hai chỉ có ý nghĩa sau khi một kết quả cụ thể của quyết định trước đã xảy ra.

Câu 2

Tại sao thực lệ đặc biệt quan trọng trong rẽ nhánh lồng nhau?

Đáp án

Vì thực lệ xác định câu lệnh bên trong thuộc nhánh nào của câu lệnh bên ngoài.

Câu 3

Mô phỏng bằng tay giúp hiểu điều gì?

Đáp án

Giúp hiểu thứ tự thực hiện, nhánh nào được chọn và giá trị các biến thay đổi như thế nào.

3.4 Nhiều phương án

Nhiều quyết định có nhiều hơn hai kết quả có thể xảy ra.

Python dùng `elif` để biểu diễn chuỗi các phương án loại trừ lẫn nhau.

Cấu trúc tổng quát:

```
if condition1 :  
    statements1  
elif condition2 :
```

```

        statements2
elif condition3 :
    statements3
else :
    defaultStatements

```

Python kiểm tra điều kiện từ trên xuống dưới. Khi gặp điều kiện đầu tiên đúng, nhánh tương ứng được thực hiện và các nhánh còn lại bị bỏ qua.

Ví dụ – Phân loại động đất

Một cách phân loại đơn giản có thể dùng các ngưỡng:

```

richter = float(input("Enter a magnitude: "))

if richter >= 8.0 :
    description = "Very severe structural damage"
elif richter >= 7.0 :
    description = "Severe damage in many areas"
elif richter >= 6.0 :
    description = "Considerable damage is possible"
elif richter >= 4.5 :
    description = "Some poorly constructed buildings may be damaged"
else :
    description = "Little or no structural damage"

print(description)

```

Điểm cần chú ý là cấu trúc rẽ nhánh, không phải nội dung cụ thể của các mô tả.

3.4.1 Thứ tự điều kiện rất quan trọng

Giả sử điều kiện được viết từ ngưỡng nhỏ lên ngưỡng lớn:

```

if richter >= 4.5 :
    ...
elif richter >= 6.0 :
    ...
elif richter >= 7.0 :
    ...

```

Với richter = 7.1, điều kiện đầu tiên đã đúng nên các nhánh sau không bao giờ được kiểm tra.

Khi các khoảng chồng lấn, cần kiểm tra **ngưỡng cao hơn / trường hợp cụ thể hơn trước**.

Mẫu đúng:

```

if value >= 80 :
    ...
elif value >= 70 :
    ...
elif value >= 60 :
    ...
else :
    ...

```

3.4.2 elif và các câu lệnh if độc lập

Hai cách này không tương đương.

Các phương án loại trừ lẫn nhau:

```
if score >= 90 :  
    grade = "A"  
elif score >= 80 :  
    grade = "B"  
elif score >= 70 :  
    grade = "C"
```

Chỉ một nhánh được thực hiện.

Các kiểm tra độc lập:

```
if score >= 90 :  
    print("At least 90")  
if score >= 80 :  
    print("At least 80")  
if score >= 70 :  
    print("At least 70")
```

Với score = 95, cả ba dòng đều được in.

Dùng if/elif/else khi các trường hợp loại trừ lẫn nhau. Dùng nhiều if độc lập khi nhiều điều kiện có thể đồng thời kích hoạt nhiều hành động.

Tự kiểm tra 3.4

Câu 1

Viết logic đặt sign bằng 1, 0 hoặc -1 tùy theo x dương, bằng 0 hay âm.

Đáp án

```
if x > 0 :  
    sign = 1  
elif x < 0 :  
    sign = -1  
else :  
    sign = 0
```

Câu 2

Tại sao các điều kiện ngưỡng chồng lấn thường nên sắp xếp từ lớn xuống nhỏ?

Đáp án

Vì Python dừng ở nhánh đúng đầu tiên. Nếu đặt một điều kiện rộng ở trước, nó có thể chặn một trường hợp cụ thể hơn ở phía sau.

TOOLBOX 3.1 – Gửi e-mail

Giáo trình dùng tự động hóa e-mail để minh họa cách kết hợp dữ liệu, quyết định và thư viện Python.

Ý tưởng lập trình cốt lõi là tạo nội dung thông điệp khác nhau tùy theo điều kiện:

```
score = int(input("Score: "))
body = "Your score on the last exam is " + str(score) + "\n"

if score <= 50 :
    body += "Please consider visiting the tutoring center."
elif score >= 90 :
    body += "Excellent work."
```

Chi tiết thư viện gửi mail không phải trọng tâm chính của bài. Điều cần hiểu là chương trình có thể tùy biến hành động hoặc nội dung dựa trên dữ liệu.

Với tài khoản thật, yêu cầu xác thực của nhà cung cấp e-mail có thể thay đổi theo thời gian. Không nên đặt mật khẩu thật trực tiếp trong mã nguồn bài giảng hoặc notebook chia sẻ.

3.5 Giải quyết vấn đề: Lưu đồ

Lưu đồ (*flowchart*) mô tả trực quan luồng điều khiển của thuật toán.

Các thành phần thường gặp:

[Xư' lý / công việc]

< Nhập / xuất >

```

      / Điều kiện? \
     /              \
    Đúng             Sai
```

Lưu đồ hữu ích khi bài toán có nhiều quyết định và chưa rõ cách các nhánh liên kết với nhau.

3.5.1 Quyết định hai nhánh

```

temperature < 0 ?
  /      \
 Đứng     Sai
  |      |
print("Frozen")  tiếp tục
```

3.5.2 Nhiều phương án

```

score >= 90 ?
 /      \
Đúng     Sai
 |      |
điểm A   score >= 80 ?
          /      \
        Đúng     Sai
```

```

      |           |
điểm B      ...

```

3.5.3 Tránh luồng điều khiển kiểu “spaghetti”

Một nguyên tắc thiết kế quan trọng là:

Không vẽ mũi tên từ một nhánh chui vào giữa một nhánh khác.

Các nhánh nên được tổ chức rõ ràng: tách ra, thực hiện riêng, rồi có thể nhập lại sau đó.

Cách tổ chức này phù hợp với `if`, `if/else` và các cấu trúc lồng nhau có cấu trúc rõ ràng.

Ví dụ – Chi phí vận chuyển

Giả sử:

- vận chuyển trong phần lục địa của Hoa Kỳ có một mức giá;
- Alaska hoặc Hawaii có mức khác;
- vận chuyển quốc tế dùng mức cao hơn.

Một lời giải có cấu trúc:

```

country = input("Enter the country: ")
state = input("Enter the state or province: ")

if country == "USA" :
    if state == "AK" or state == "HI" :
        shippingCost = 10.0
    else :
        shippingCost = 5.0
else :
    shippingCost = 10.0

print("Shipping cost to %s, %s: $%.2f" %
      (state, country, shippingCost))

```

Có thể vẽ lưu đồ trước rồi mới dịch sang Python.

Tự kiểm tra 3.5

Câu 1

Mục đích chính của lưu đồ là gì?

Đáp án

Trực quan hóa các công việc, quyết định và các đường đi có thể xảy ra của chương trình trước hoặc trong khi cài đặt thuật toán.

Câu 2

Tại sao không nên để mũi tên đi vào giữa một nhánh khác?

Đáp án

Vì nó tạo ra luồng điều khiển thiếu cấu trúc, khó dịch sang chương trình, khó kiểm thử và khó bảo trì.

3.6 Giải quyết vấn đề: Ca kiểm thử

Chương trình có rẽ nhánh phải được kiểm thử bằng các đầu vào đi qua các đường thực hiện khác nhau.

Chỉ thử một trường hợp thông thường là chưa đủ.

Một kế hoạch kiểm thử tốt hướng tới:

1. **bao phủ nhánh** – mỗi nhánh được thực hiện bởi ít nhất một ca kiểm thử;
2. **kiểm thử biên** – kiểm tra các giá trị đúng tại hoặc gần ngưỡng quyết định;
3. **kiểm thử đầu vào không hợp lệ** – khi chương trình có nhiệm vụ xác thực dữ liệu.

3.6.1 Bao phủ nhánh

Với quyết định hai nhánh:

```
if age >= 18 :
    category = "Adult"
else :
    category = "Minor"
```

Cần ít nhất một ca cho mỗi kết quả:

Đầu vào	Kết quả mong đợi	Mục đích
20	Adult	Nhánh đúng
15	Minor	Nhánh sai

3.6.2 Các giá trị biên

Ngưỡng ở đây là 18.

Các ca hữu ích:

17
18
19

Giá trị 18 đặc biệt quan trọng vì giúp phân biệt $\geq$ với $>$.

3.6.3 Thiết kế ca kiểm thử trước khi lập trình

Viết trước kết quả mong đợi có nhiều lợi ích:

- buộc ta hiểu rõ quy tắc;
- phát hiện điều kiện biên chưa rõ ràng;
- cho phép kiểm chứng chương trình ngay sau khi cài đặt;
- tránh xu hướng chấp nhận mọi kết quả mà chương trình tình cờ in ra.

Mẫu bảng kiểm thử:

Ca kiểm thử	Kết quả mong đợi	Lý do
trường hợp thường A	...	nhánh thứ nhất
trường hợp thường B	...	nhánh thứ hai
giá trị biên	...	đúng tại ngưỡng
giá trị không hợp lệ	báo lỗi	kiểm tra validation

Mẹo lập trình 3.3 – Dành thời gian cho kiểm thử và gỡ lỗi

Lập trình không chỉ là gõ mã. Một kế hoạch thực tế cần có thời gian cho:

- hiểu và thiết kế lời giải;
- chuẩn bị ca kiểm thử;
- nhập chương trình và sửa lỗi cú pháp;
- chạy thử và gỡ lỗi hành vi không mong đợi.

Chương trình càng nhiều nhánh thì càng có nhiều đường thực hiện và thường cần nhiều công sức kiểm thử hơn.

Tự kiểm tra 3.6

Câu 1

Với điều kiện:

```
if value < 100 :
```

giá trị biên quan trọng là gì?

Đáp án

100. Các giá trị 99 và 101 cũng hữu ích để kiểm tra lân cận ngưỡng.

Câu 2

Tại sao một ca kiểm thử thường không đủ cho `if/else`?

Đáp án

Vì nó có thể chỉ thực hiện một nhánh và để nhánh còn lại chưa từng được kiểm tra.

3.7 Biến Boolean và các toán tử Boolean

Kiểu Boolean `bool` chỉ có hai giá trị:

`True`

`False`

Một biến Boolean có thể lưu kết quả của một điều kiện logic.

```
failed = True
```

```
if failed :
    print("The operation failed")
```

Biến Boolean đôi khi được gọi là **cờ** (*flag*) vì trạng thái của nó biểu diễn một điều kiện đang đúng hay sai.

3.7.1 Toán tử and

and chỉ đúng khi **cả hai** toán hạng đều đúng.

```
if temp > 0 and temp < 100 :
    print("Liquid water")
```

Bảng chân trị:

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

3.7.2 Toán tử or

or đúng khi **ít nhất một** toán hạng đúng.

```
if temp <= 0 or temp >= 100 :
    print("Not liquid water")
```

Bảng chân trị:

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

Lưu ý: or là phép “hoặc bao hàm”, tức là vẫn đúng khi **cả hai** điều kiện đều đúng.

3.7.3 Toán tử not

not đảo giá trị Boolean.

```
if not frozen :
    print("Not frozen")
```

A	not A
True	False
False	True

3.7.4 Thứ tự ưu tiên

Một thứ tự đơn giản để ghi nhớ:

sô' học
↓
toán tư' quan hệ
↓
not
↓
and
↓
or

Do đó:

```
x > 0 and y < 10
```

được hiểu là:

```
(x > 0) and (y < 10)
```

Với biểu thức logic phức tạp, dấu ngoặc vẫn nên được dùng để tăng tính dễ đọc.

Lỗi thường gặp 3.3 – Nhầm and với or

Giả sử một giá trị hợp lệ khi nằm trong khoảng từ 0 đến 100.

Đúng:

```
if value >= 0 and value <= 100 :  
    print("Valid")
```

Cả hai điều kiện phải cùng đúng.

Để phát hiện giá trị không hợp lệ:

```
if value < 0 or value > 100 :  
    print("Invalid")
```

Chỉ cần một trong các điều kiện không hợp lệ là đúng.

Câu hỏi gợi ý:

- Có cần **tất cả** điều kiện cùng đúng không? → dùng and.
 - Chỉ cần **một** điều kiện đúng là đủ không? → dùng or.
-

Mẹo lập trình 3.4 – Tính dễ đọc

Không nên so sánh biến Boolean một cách thừa thãi với True hoặc False.

Kém rõ ràng:

```
if frozen == False :  
    print("Not frozen")
```

Nên viết:

```
if not frozen :  
    print("Not frozen")
```

Tương tự:

```
if valid :  
    print("Input accepted")
```

thường dễ đọc hơn:

```
if valid == True :  
    print("Input accepted")
```

Tên biến Boolean nên mô tả rõ một trạng thái hoặc điều kiện:

```
valid  
finished  
found  
isEmpty  
hasError
```

Chủ đề bổ sung 3.3 – Nối chuỗi các phép so sánh

Python cho phép viết:

```
0 <= value <= 100
```

Tương đương với:

```
value >= 0 and value <= 100
```

Cách nối này gọn và tự nhiên trong Python, nhưng người mới học nên hiểu rõ biểu thức Boolean tương đương.

Chủ đề bổ sung 3.4 – Đánh giá ngắn mạch

Python đánh giá and và or từ trái sang phải và dừng ngay khi đã xác định được kết quả cuối cùng.

Ví dụ:

```
quantity > 0 and price / quantity < 10
```

Nếu `quantity > 0` là sai, Python không thực hiện phép chia. Điều này tránh chia cho 0 khi `quantity == 0`.

Với or:

```
conditionA or conditionB
```

nếu `conditionA` đã đúng, `conditionB` sẽ không được đánh giá.

Cơ chế này gọi là **đánh giá ngắn mạch** (*short-circuit evaluation*).

Chủ đề bổ sung 3.5 – Định luật De Morgan

Định luật De Morgan giúp biến đổi các điều kiện ghép có phủ định.

`not (A and B) ≡ (not A) or (not B)`

`not (A or B) ≡ (not A) and (not B)`

Ví dụ:

```
not (state == "AK" or state == "HI")
```

tương đương với:

```
state != "AK" and state != "HI"
```

Ví dụ khác:

```
not (country == "USA" and state != "AK" and state != "HI")
```

có thể viết lại trực tiếp hơn:

```
country != "USA" or state == "AK" or state == "HI"
```

Mục tiêu không chỉ là viết ngắn hơn mà là làm logic rõ ràng hơn.

Tự kiểm tra 3.7

Câu 1

Làm thế nào kiểm tra cả x và y đều dương?

Đáp án

```
x > 0 and y > 0
```

Câu 2

Làm thế nào kiểm tra ít nhất một trong x, y bằng 0?

Đáp án

```
x == 0 or y == 0
```

Câu 3

Giá trị của:

```
not not True
```

là gì?

Đáp án

True.

Câu 4

Tại sao biểu thức sau vẫn an toàn khi `quantity == 0`?

```
quantity > 0 and price / quantity < 10
```

Đáp án

Vì đánh giá ngắn mạch dừng ngay sau điều kiện đầu tiên sai, nên phép chia không được thực hiện.

3.8 Phân tích chuỗi

Logic rẽ nhánh thường được dùng để kiểm tra dữ liệu văn bản.

Python cung cấp nhiều toán tử và phương thức để xác định một chuỗi có chứa chuỗi con hay có đặc điểm nhất định hay không.

3.8.1 Toán tử thành viên `in` và `not in`

```
name = "John Wayne"
```

```
if "Way" in name :
    print("Substring found")
```

Kiểm tra ngược lại:

```
if "-" not in name :
    print("The name does not contain a hyphen")
```

Kiểm tra thành viên phân biệt chữ hoa/chữ thường.

3.8.2 Một số phương thức chuỗi con hữu ích

Phép toán	Ý nghĩa
<code>substring in s</code>	s có chứa substring hay không?
<code>s.count(substring)</code>	Số lần xuất hiện không chồng lấn
<code>s.startswith(substring)</code>	s có bắt đầu bằng chuỗi con không?
<code>s.endswith(substring)</code>	s có kết thúc bằng chuỗi con không?
<code>s.find(substring)</code>	Chỉ số lần xuất hiện đầu tiên, hoặc -1 nếu không tìm thấy

Ví dụ:

```
filename = "report.html"
```

```
if filename.endswith(".html") :
    print("HTML file")
```

Ví dụ:

```
text = "banana"
print(text.count("an"))
print(text.find("na"))
```

3.8.3 Kiểm tra đặc điểm của chuỗi

Một số phương thức hữu ích:

Phương thức	Trả về True khi...
<code>s.isalnum()</code>	mọi ký tự đều là chữ hoặc số và chuỗi không rỗng
<code>s.isalpha()</code>	mọi ký tự đều là chữ cái và chuỗi không rỗng

Phương thức	Trả về True khi...
<code>s.isdigit()</code>	mọi ký tự đều là chữ số và chuỗi không rỗng
<code>s.islower()</code>	có ít nhất một chữ cái và mọi chữ cái đều viết thường
<code>s.isupper()</code>	có ít nhất một chữ cái và mọi chữ cái đều viết hoa
<code>s.isspace()</code>	mọi ký tự đều là khoảng trắng và chuỗi không rỗng

Ví dụ:

```
"1729".isdigit()
```

trả về True.

```
"-1729".isdigit()
```

trả về False vì - không phải chữ số.

```
"John Smith".isalpha()
```

trả về False vì chuỗi chứa khoảng trắng.

Ví dụ – Khảo sát chuỗi con

```
theString = input("Enter a string: ")
theSubstring = input("Enter a substring: ")

if theSubstring in theString :
    print("The string contains the substring.")
    print("Count:", theString.count(theSubstring))
    print("First position:", theString.find(theSubstring))

    if theString.startswith(theSubstring) :
        print("It appears at the beginning.")

    if theString.endswith(theSubstring) :
        print("It appears at the end.")
else :
    print("The string does not contain the substring.")
```

Tự kiểm tra 3.8

Câu 1

Làm thế nào để đếm số khoảng trắng trong text?

Đáp án

```
text.count(" ")
```

Câu 2

Làm thế nào để kiểm tra tên file kết thúc bằng .jpg hoặc .jpeg?

Đáp án

```
filename.endswith(".jpg") or filename.endswith(".jpeg")
```

Nếu muốn không phân biệt chữ hoa/thường, hãy chuẩn hóa bằng `lower()` trước.

Câu 3

`find()` trả về gì khi không tìm thấy chuỗi con?

Đáp án

-1.

3.9 Ứng dụng: Kiểm tra dữ liệu đầu vào

Kiểm tra dữ liệu đầu vào (*input validation*) là xác minh dữ liệu do người dùng cung cấp trước khi sử dụng.

Chương trình không nên giả định mọi dữ liệu nhập đều thỏa mãn yêu cầu.

Giả sử thang máy chấp nhận các tầng từ 1 đến 20, trừ tầng 13.

Các đầu vào không hợp lệ gồm:

- 13;
- 0 hoặc số âm;
- số lớn hơn 20.

Phiên bản có validation:

```
floor = int(input("Floor: "))

if floor == 13 :
    print("Error: There is no thirteenth floor.")
elif floor <= 0 or floor > 20 :
    print("Error: The floor must be between 1 and 20.")
else :
    actualFloor = floor

    if floor > 13 :
        actualFloor = floor - 1

    print("The elevator will travel to the actual floor", actualFloor)
```

Mẫu tổng quát:

Đọc dữ liệu

↓

Kiểm tra hợp lệ

↓

Không hợp lệ? → báo lỗi

Hợp lệ? → xử lý

3.9.1 Kiểm tra miền giá trị

Ví dụ:

```
if age < 0 or age > 130 :
    print("Invalid age")
else :
```

```
# Có thể xử lý giá trị an toàn hơn.  
...
```

3.9.2 Kiểm tra lựa chọn nhập vào

Giả sử người dùng phải nhập s hoặc m.

Một cách:

```
status = input("Enter s or m: ")  
  
if status == "s" or status == "S":  
    ...  
elif status == "m" or status == "M":  
    ...  
else:  
    print("Invalid status")
```

Cách dễ mở rộng hơn là chuẩn hóa trước:

```
status = input("Enter s or m: ")  
status = status.lower()
```

```
if status == "s":  
    ...  
elif status == "m":  
    ...  
else:  
    print("Invalid status")
```

Tương tự, mã nhiều ký tự có thể chuẩn hóa bằng:

```
country = input("Country code: ")  
country = country.upper()
```

Sau đó chỉ cần so sánh với một dạng chuẩn.

3.9.3 Giới hạn ở giai đoạn hiện tại

Đoạn mã:

```
floor = int(input("Floor: "))
```

vẫn phát sinh ngoại lệ nếu người dùng nhập:

```
five
```

Xử lý lỗi chuyển đổi kiểu cần cơ chế exception, được giới thiệu ở chương sau.

Ở giai đoạn này cần phân biệt:

- **kiểm tra giá trị sau khi chuyển đổi** – ví dụ kiểm tra có nằm trong khoảng hợp lệ không;
 - **xử lý việc chuyển đổi thất bại** – chủ đề về ngoại lệ học ở phần sau.
-

Chủ đề bổ sung 3.6 – Kết thúc chương trình

Với chương trình văn bản nhỏ, dữ liệu sai đôi khi có thể xử lý bằng cách kết thúc chương trình ngay.

```
from sys import exit

response = input("Enter y or n: ")

if not (response == "y" or response == "n") :
    exit("Error: you must enter y or n.")
```

Sau khi validation thành công, phần mã phía sau có thể giả định đầu vào đã thỏa điều kiện.

Không nên lạm dụng việc kết thúc sớm. Trong chương trình lớn hơn, hàm và ngoại lệ thường giúp tổ chức mã tốt hơn.

Chủ đề bổ sung 3.7 – Chương trình đồ họa tương tác

Logic rẽ nhánh cũng áp dụng trong chương trình đồ họa.

Một chương trình có thể:

1. đọc dữ liệu;
2. kiểm tra tọa độ hoặc kích thước;
3. chỉ vẽ khi dữ liệu hợp lệ;
4. phản ứng với một cú nhấp chuột và rẽ nhánh theo vị trí nhấp.

Điểm quan trọng là giao diện đồ họa không làm mất nhu cầu kiểm tra dữ liệu đầu vào.

Máy tính và xã hội 3.2 – Trí tuệ nhân tạo

Chương giới thiệu trí tuệ nhân tạo như một bối cảnh rộng hơn cho phần mềm ra quyết định. Chương trình truyền thống tuân theo các quy tắc do lập trình viên mô tả rõ, trong khi AI hướng đến những hệ thống có hành vi linh hoạt hoặc thông minh hơn.

Liên hệ với bài học này chủ yếu mang tính khái niệm: ngay cả các hệ thống phức tạp vẫn phải biểu diễn thông tin, đánh giá phương án và chọn hành động. Các kỹ thuật AI hiện đại có thể học một phần quy tắc quyết định từ dữ liệu, nhưng hiểu lường điều khiển tường minh vẫn là nền tảng của lập trình.

Ví dụ minh họa 3.2 – Hai đường tròn giao nhau

Ví dụ đồ họa của chương xác định quan hệ giữa hai đường tròn.

Giả sử tâm hai đường tròn là:

(x_0, y_0)
 (x_1, y_1)

và bán kính:

r_0
 r_1

Khoảng cách giữa hai tâm:

```
from math import sqrt
```

```
dist = sqrt((x1 - x0) ** 2 + (y1 - y0) ** 2)
```

Có thể phân loại quan hệ bằng rẽ nhánh:

```
if dist > r0 + r1 :
    message = "The circles are separate."
elif dist < abs(r0 - r1) :
    message = "One circle is inside the other."
elif dist == r0 + r1 :
    message = "The circles touch externally."
elif dist == 0 and r0 == r1 :
    message = "The circles coincide."
else :
    message = "The circles intersect at two points."
```

Ví dụ này kết hợp:

- kiểm tra đầu vào;
- số học;
- toán tử quan hệ;
- toán tử Boolean;
- nhiều phương án;
- hình học;
- đầu ra đồ họa.

Với các phép tính hình học dùng float, nên cân nhắc kiểm tra gần bằng thay vì bằng tuyệt đối.

TOOLBOX 3.2 – Vẽ đồ thị đơn giản

Chương giới thiệu matplotlib.pyplot như một ví dụ tùy chọn về thư viện Python dùng để trực quan hóa dữ liệu.

Biểu đồ cột đơn giản:

```
from matplotlib import pyplot
```

```
pyplot.bar(1, 1.1)
pyplot.bar(2, 10.0)
pyplot.bar(3, 25.4)
pyplot.bar(4, 44.5)
pyplot.bar(5, 61.0)
```

```
pyplot.xlabel("Month")
pyplot.ylabel("Temperature")
pyplot.show()
```

Đồ thị đường đơn giản:

```
from matplotlib import pyplot
```

```
months = [1, 2, 3, 4, 5]
temperatures = [1.1, 10.0, 25.4, 44.5, 61.0]
```

```
pyplot.plot(months, temperatures)
```

```
pyplot.xlabel("Month")
pyplot.ylabel("Temperature")
pyplot.show()
```

Toolbox về đồ thị là phần tùy chọn. Mục tiêu chính của chương vẫn là câu lệnh rẽ nhánh và logic Boolean.

Tổng kết Bài 3

Câu lệnh `if`

- `if` thay đổi đường thực hiện của chương trình dựa trên một điều kiện.
- `else` cung cấp phương án thay thế khi điều kiện sai.
- `elif` hỗ trợ nhiều phương án loại trừ lẫn nhau.
- Câu lệnh ghép có dòng tiêu đề và một khối thụt lề.
- Thụt lề nhất quán là một phần của cú pháp Python.
- Tránh lặp mã giữa các nhánh khi phần chung có thể đưa ra ngoài.

Các toán tử quan hệ

`<` `<=` `>` `>=` `==` `!=`

Ghi nhớ:

`=` gán

`==` kiểm tra bằng nhau

Chuỗi cũng có thể được so sánh và phép so sánh mặc định phân biệt chữ hoa/chữ thường.

Số thực được tạo từ phép tính thường nên so sánh bằng sai số cho phép thay vì bằng tuyệt đối.

Rẽ nhánh lồng nhau

- Một câu lệnh `if` có thể nằm trong một nhánh khác.
- Rẽ nhánh lồng nhau phù hợp với quyết định nhiều tầng.
- Thụt lề cho biết quyết định bên trong thuộc nhánh nào.
- Mô phỏng bằng tay giúp kiểm tra logic lồng nhau.

Nhiều phương án

Dùng:

```
if ... :
    ...
elif ... :
    ...
else :
    ...
```

Chỉ nhánh đúng đầu tiên được thực hiện.

Với các ngưỡng chồng lấn, phải sắp xếp điều kiện cẩn thận.

Lưu đồ

Lưu đồ biểu diễn trực quan:

- công việc xử lý;

- nhập/xuất;
- quyết định;
- nhánh và điểm nhập lại.

Nên dùng cấu trúc rõ ràng và tránh các mũi tên “nhảy” vào giữa nhánh khác.

Ca kiểm thử

Ca kiểm thử tốt nên gồm:

- ít nhất một ca cho mỗi nhánh;
- giá trị biên;
- các giá trị ngay bên dưới và bên trên biên;
- giá trị không hợp lệ khi có yêu cầu validation.

Thiết kế kết quả mong đợi trước khi viết mã giúp cả việc xây dựng thuật toán lẫn kiểm thử.

Logic Boolean

True
False
and
or
not

- and yêu cầu tất cả điều kiện thành phần đúng.
- or yêu cầu ít nhất một điều kiện đúng.
- not đảo giá trị Boolean.
- Python sử dụng đánh giá ngắn mạch.
- Định luật De Morgan giúp biến đổi điều kiện phủ định phức tạp.

Phân tích chuỗi

Các phép toán hữu ích:

```
substring in text  
substring not in text  
text.count(...)  
text.find(...)  
text.startswith(...)  
text.endswith(...)  
text.isalpha()  
text.isdigit()  
text.isalnum()  
text.islower()  
text.isupper()  
text.isspace()
```

Kiểm tra dữ liệu đầu vào

Trước khi xử lý dữ liệu do người dùng nhập:

Đọc

- chuẩn hóa nếu cần
- kiểm tra hợp lệ
- từ chối giá trị sai
- xử lý giá trị đúng

Kiểm tra miền giá trị và các lựa chọn hợp lệ có thể làm bằng điều kiện. Lỗi chuyển đổi số cần xử lý ngoại lệ, sẽ học sau.

Quiz tổng kết

Câu 1

Toán tử nào kiểm tra bằng nhau?

- A. =
- B. ==
- C. !=
- D. <=

Đáp án

B. ==

Câu 2

Điều gì xảy ra khi điều kiện của `if/else` là đúng?

- A. Cả hai nhánh đều chạy.
- B. Chỉ nhánh `else` chạy.
- C. Nhánh `if` chạy và `else` bị bỏ qua.
- D. Chương trình luôn kết thúc.

Đáp án

C

Câu 3

Giá trị của:

`5 >= 5`

là gì?

- A. `True`
- B. `False`
- C. `5`
- D. Lỗi

Đáp án

A. `True`

Câu 4

Điều kiện nào kiểm tra `x` nằm *nghiêm ngặt* giữa `0` và `10`?

- A. `x > 0 or x < 10`
- B. `x > 0 and x < 10`
- C. `x >= 0 or x <= 10`
- D. `not x`

Đáp án

B

Câu 5

Kết quả của:

`True or False`

là gì?

Đáp án

True.

Câu 6

Tại sao phép kiểm tra sau có thể không an toàn sau các phép tính số thực?

`if result == expected :`

Đáp án

Vì sai số làm tròn nhỏ có thể khiến hai giá trị về mặt toán học bằng nhau nhưng biểu diễn float hơi khác nhau. Trong nhiều trường hợp nên so sánh theo một ngưỡng sai số.

Câu 7

Cấu trúc nào thường phù hợp nhất cho các loại điểm loại trừ lẫn nhau?

- A. Nhiều `if` độc lập
- B. `if/elif/else`
- C. Chỉ `else`
- D. Không cần điều kiện

Đáp án

B. `if/elif/else`

Câu 8

Biểu thức sau kiểm tra điều gì?

`".csv" in filename`

Đáp án

Kiểm tra chuỗi con chính xác `".csv"` có xuất hiện ở bất kỳ vị trí nào trong `filename` hay không.

Câu 9

`text.find("abc")` trả về gì khi không có `"abc"`?

- A. False
- B. None
- C. -1
- D. Lỗi

Đáp án

C. -1

Câu 10

Giá trị biên cho:

```
if score >= 50 :
```

là gì?

Đáp án

50 là giá trị biên quan trọng. 49 và 51 là hai giá trị lân cận hữu ích.

Câu 11

Tại sao thứ tự điều kiện trong chuỗi if/elif có thể ảnh hưởng kết quả?

Đáp án

Vì Python dừng ở nhánh đúng đầu tiên. Một điều kiện quá rộng đặt quá sớm có thể khiến điều kiện cụ thể hơn ở phía sau không bao giờ được kiểm tra.

Câu 12

Mục tiêu chính của kiểm tra dữ liệu đầu vào là gì?

- A. Làm mã dài hơn.
- B. Đảm bảo dữ liệu người dùng nhập thỏa các giả định của chương trình trước khi xử lý.
- C. Chuyển mọi giá trị thành chuỗi.
- D. Loại bỏ tất cả câu lệnh if.

Đáp án

B

Bài tập ôn tập**Bài 1. Theo dõi quyết định hai nhánh**

Xác định giá trị cuối cùng của fee:

```
age = 16
```

```
if age < 18 :  
    fee = 5  
else :  
    fee = 10
```

Đáp án

fee = 5.

Bài 2. Chọn đúng toán tử quan hệ

Điền toán tử để thông báo được in khi temperature không lớn hơn 0.

```
if temperature ____ 0 :  
    print("Freezing or below")
```

Đáp án

<=

Bài 3. Viết lại để tránh lặp mã

Cải thiện đoạn sau:

```

if quantity >= 10 :
    rate = 0.90
    total = rate * price * quantity
else :
    rate = 1.00
    total = rate * price * quantity

```

Đáp án tham khảo

```

if quantity >= 10 :
    rate = 0.90
else :
    rate = 1.00

total = rate * price * quantity

```

Bài 4. Thiết kế ca kiểm thử

Với:

```

if balance < 0 :
    status = "Overdrawn"
else :
    status = "OK"

```

hãy đề xuất ba giá trị kiểm thử hữu ích.

Đáp án tham khảo

```

-1  → Overdrawn
0   → OK        (biên)
1   → OK

```

Bài 5. Biểu thức Boolean

Viết điều kiện cho các trường hợp:

1. x và y đều dương.
2. Ít nhất một trong x, y âm.
3. age nằm trong khoảng từ 18 đến 65, kể cả hai đầu.
4. answer không phải "y" cũng không phải "n".

Đáp án tham khảo

```

x > 0 and y > 0
x < 0 or y < 0
age >= 18 and age <= 65
answer != "y" and answer != "n"

```

Bài 6. Phân tích chuỗi

Cho:

```
filename = "report.Final.PDF"
```

viết điều kiện không phân biệt chữ hoa/thường để kiểm tra file có kết thúc bằng .pdf hay không.

Đáp án

```
filename.lower().endswith(".pdf")
```

Bài tập thực hành

Bài 1. Đạt / Không đạt / Điểm không hợp lệ

Nhập một điểm thi.

- Nếu điểm nhỏ hơn 0 hoặc lớn hơn 100, in thông báo lỗi.
- Nếu hợp lệ, in "Pass" cho điểm từ 50 trở lên và "Fail" cho điểm thấp hơn.

Hãy vẽ lưu đồ trước khi viết Python.

Bài 2. Số lớn hơn trong hai số

Nhập hai số và in:

- số lớn hơn;
- "Equal" nếu hai số bằng nhau.

Dùng if/elif/else.

Bài 3. Số lớn nhất trong ba số

Nhập ba số và xác định số lớn nhất.

Trước tiên giải bằng rẽ nhánh lồng nhau. Sau đó xem có thể viết rõ hơn bằng biểu thức Boolean hoặc nhiều phương án hay không.

Bài 4. Quy tắc vận chuyển

Nhập:

- quốc gia đích;
- bang hoặc tỉnh.

Chuẩn hóa hai đầu vào bằng upper().

Dùng quy tắc:

- phần lục địa Hoa Kỳ: \$5;
- Alaska hoặc Hawaii: \$10;
- ngoài Hoa Kỳ: \$15.

In phí vận chuyển với hai chữ số thập phân.

Bài 5. Phân loại điểm chữ

Nhập điểm từ 0 đến 100 và phân loại:

90–100 → A
80–89 → B
70–79 → C
60–69 → D
0–59 → F

Từ chối điểm không hợp lệ trước khi phân loại.

Tạo ca kiểm thử cho:

-1, 0, 59, 60, 69, 70, 79, 80, 89, 90, 100, 101

Bài 6. Kiểm tra tên người dùng

Nhập một username và kiểm tra:

- không được rỗng;
- chỉ gồm chữ cái và chữ số;
- có ít nhất một ký tự.

Dùng các phương thức phân tích chuỗi ở Mục 3.8.

Bài 7. Phân loại loại file

Nhập tên file và phân loại:

- mã Python: .py;
- notebook: .ipynb;
- dữ liệu CSV: .csv;
- file văn bản: .txt;
- loại khác.

Kiểm tra không phân biệt chữ hoa/chữ thường.

Bài 8. So sánh gần bằng

Nhập hai số thực và xác định chúng có chênh nhau ít hơn 0.001 hay không.

Không dùng phép bằng tuyệt đối cho phép so sánh gần bằng.

Bài 9. Phân loại tam giác theo độ dài cạnh

Nhập ba độ dài cạnh dương.

Trước tiên kiểm tra chúng có tạo thành tam giác hay không:

```
a + b > c
and
b + c > a
and
c + a > b
```

Nếu hợp lệ, phân loại:

- tam giác đều;
- tam giác cân;
- tam giác thường.

Chuẩn bị ca kiểm thử cho từng nhánh và các giá trị biên trước khi viết mã.

Bài 10. Kiểm tra mã truy cập đơn giản

Nhập một mã truy cập ngắn.

Quy tắc:

- đúng 6 ký tự;
- chỉ gồm chữ và số;
- phải chứa chuỗi con "NEU", không phân biệt hoa/thường.

In thông báo cụ thể cho trường hợp hợp lệ hoặc không hợp lệ.

Bài tập mở

Bài 1

Đưa ra ba ví dụ thực tế trong đó một quyết định chỉ có đúng hai kết quả. Viết quy tắc cho mỗi ví dụ bằng mã giả.

Bài 2

Đưa ra một ví dụ thực tế có ít nhất bốn phương án loại trừ lẫn nhau. Giải thích vì sao `if/elif/else` phù hợp.

Bài 3

Chọn một điều kiện dạng khoảng và xác định:

- một giá trị hợp lệ điển hình;
- giá trị biên dưới;
- giá trị biên trên;
- một giá trị ngay dưới khoảng;
- một giá trị ngay trên khoảng.

Bài 4

Tạo ví dụ trong đó nhiều `if` độc lập là đúng nhưng `if/elif` lại sai. Giải thích vì sao có thể cần thực hiện nhiều hành động cùng lúc.

Bài 5

Viết một điều kiện Boolean phức tạp dùng `not`, `and`, `or`. Sau đó đơn giản hóa bằng định luật De Morgan.

Thuật ngữ chính

Thuật ngữ	Ý nghĩa
Decision	Quyết định – chọn hành động dựa trên điều kiện
Condition	Điều kiện – biểu thức cho kết quả đúng hoặc sai
<code>if</code> statement	Câu lệnh <code>if</code> – thực hiện một khối khi điều kiện đúng
<code>else</code> branch	Nhánh thay thế khi điều kiện <code>if</code> sai
<code>elif</code>	Thêm một điều kiện khác vào quyết định nhiều nhánh
Compound statement	Câu lệnh ghép gồm dòng tiêu đề và các câu lệnh lồng bên dưới
Statement block	Nhóm câu lệnh ở cùng mức thụt lề
Indentation	Khoảng trắng đầu dòng xác định cấu trúc khối trong Python
Relational operator	Toán tử quan hệ so sánh hai giá trị
Equality test	Kiểm tra bằng nhau bằng <code>==</code>
Boundary value	Giá trị nằm đúng tại biên của một khoảng quyết định
Floating-point tolerance	Sai số cho phép dùng khi so sánh gần bằng
Nested branch	Câu lệnh quyết định nằm trong một nhánh khác
Hand-tracing	Mô phỏng thủ công quá trình chạy chương trình
Multiple alternatives	Quyết định có nhiều hơn hai nhánh
Flowchart	Sơ đồ mô tả công việc và luồng điều khiển
Branch coverage	Kiểm thử để mọi nhánh đều được chạy ít nhất một lần
Test case	Đầu vào cùng kết quả mong đợi để kiểm chứng chương trình
Boolean	Kiểu dữ liệu có hai giá trị <code>True</code> và <code>False</code>
Flag	Biến Boolean biểu diễn trạng thái hoặc điều kiện
Boolean operator	<code>and</code> , <code>or</code> , <code>not</code>
Short-circuit evaluation	Dừng đánh giá Boolean khi đã biết kết quả cuối
De Morgan's laws	Quy tắc biến đổi phủ định của biểu thức <code>and/or</code>
Substring	Chuỗi con nằm trong một chuỗi khác
Membership test	Kiểm tra bằng <code>in</code> hoặc <code>not in</code>
Input validation	Kiểm tra dữ liệu đầu vào thỏa các ràng buộc
Normalization	Chuyển dữ liệu về một dạng thống nhất trước khi so sánh

Hướng dẫn học tập

Sau Bài 3, sinh viên không nên còn xem chương trình chỉ là một danh sách phép tính cố định. Chương trình giờ đã có thể **chọn đường thực hiện**.

Một quy trình phù hợp cho bài toán có rẽ nhánh là:

Hiểu quy tắc

→ xác định các kết quả có thể xảy ra

→ xây dựng điều kiện

- kiểm tra giá trị biên
- vẽ lưu đồ khi cần
- viết mã giả
- chọn if / if-else / if-elif-else / if lồng nhau
- loại bỏ phân xử lý lặp lại
- chuẩn bị ca kiểm thử cho mọi nhánh
- cài đặt bằng Python
- chạy ca thường, ca biên và ca không hợp lệ
- mô phỏng khi kết quả bất thường

Một chu trình thực hành hữu ích:

Dự đoán → Mô phỏng → Chạy → So sánh → Giải thích → Sửa đổi → Kiểm tra biên → Luyện tập

Thói quen quan trọng nhất của chương này là coi mỗi quyết định là một cấu trúc cần được **thiết kế và kiểm thử**, không chỉ đơn thuần là gõ vài câu lệnh if.